

Musterlösungen

01 | b) Eine Datenstruktur, bei der das erste eingefügte Element als erstes entfernt wird (FIFO).

Eine Queue funktioniert nach dem FIFO-Prinzip (First In, First Out), was bedeutet, dass das erste hinzugefügte Element als erstes entfernt wird.

02 | c) Mit einer LinkedList.

Eine Queue wird in Java häufig mit einer LinkedList implementiert, da sie eine dynamische Datenstruktur ist und sowohl das Hinzufügen als auch das Entfernen von Elementen effizient unterstützt.

03 | b) FIFO (First-In-First-Out).

Das grundlegende Prinzip einer Queue ist FIFO (First In, First Out), bei dem das erste eingefügte Element als erstes entfernt wird.

04 | a) `enqueue()`

In einer Queue wird ein Element mit der Methode `enqueue()` hinzugefügt. Diese Methode fügt das Element ans Ende der Queue an.

05 | c) `dequeue()`

Mit der Methode `dequeue()` wird das erste Element aus einer Queue entfernt. Diese Methode folgt dem FIFO-Prinzip.

06 | a) `peek()`

Mit der Methode `peek()` kann das erste Element einer Queue angezeigt werden, ohne es zu entfernen.

07 | a) Es wird null zurückgegeben.

Wenn versucht wird, ein Element von einer leeren Queue zu entfernen, wird `null` zurückgegeben (wenn die `poll()`-Methode verwendet wird). Eine andere Methode wie `dequeue()` könnte eine Ausnahme werfen, wenn sie von einer leeren Queue aufgerufen wird.

08 | a) `Queue<Integer> queue = new LinkedList<Integer>();`

Eine Queue kann als LinkedList in Java deklariert werden. Die LinkedList ist eine Implementierung des `Queue`-Interfaces und stellt alle notwendigen Methoden zur Verfügung.

09 | a) isEmpty()

Mit der Methode `isEmpty()` wird überprüft, ob eine Queue leer ist.

10 | b) Ein `QueueOverflowException` wird geworfen.

Wenn versucht wird, ein Element in eine voll gefüllte Queue hinzuzufügen (bei Verwendung eines fixen Arrays als Basis), wird eine `QueueOverflowException` geworfen. Bei einer LinkedList-basierte Queue ist dies jedoch in der Regel nicht der Fall, da sie dynamisch wächst.
